Matière : Programmation O.O avec Java & Java Avancée

Enseignant/e(s) : Ammor Fatimazahra, Atigui Souad, Honnit Bouchra, Nasri Mohamed, Marzouk Abderrahim, Erraissi Allae

L'entreprise **GoAuto** souhaite créer une application java pour gérer son parc de véhicules. Elle est spécialisée dans la vente de deux types de véhicules d'occasion (Voiture, Camion). Elle possède un parking qui peut supporter 24 véhicules de type Voiture et 15 véhicules de type Camion.

Pour ce faire, il faut créer les classes suivantes :

1- Classe « **OutOfCapacityException** » qui hérite de la classe Exception et qui est levée si on atteint la capacité maximale du parc d'autos au moment de l'ajout d'un nouveau véhicule. ***(1pt)***

2- Classe abstraite « **Vehicule** » qui implémente l'interface « **Comparator** » et possède :
   - Des attributs privés : **immatriculation** *(chaîne de caractères)*, **nom** *(chaîne de caractère)*, **année de fabrication** *(entier)*, **marque** *(chaîne de caractère)*, **nombre de chevaux** *(entier)*, **carburant** *(chaîne de caractère)*, ***(2pts)***
   - Un constructeur avec paramètres ***(0.5pt)***
   - Une méthode ***toString*** qui retourne une chaîne représentative des informations ***(0.5pt)***
   - Une méthode ***compare*** de l'interface ***Comparator***. Cette méthode doit prendre en paramètres deux objets de type Vehicule et les comparer. La méthode *retourne* :
     - 1 : si l'immatricule du premier objet est plus grand (selon l'ordre alphabétique) que celui du deuxième objet ;
     - -1 : si le contraire
     - 0 : si les deux objets possèdent le même immatricule

     (1pt)
   - Une méthode abstraite ***equals*** ***(0.5pt)***

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

3- Classe « **Voiture** » qui hérite de la classe Vehicule. Cette classe possède :

- Un attribut privé supplémentaire : **nombre de sièges** *(entier)* ***(0.5pt)***
- Un constructeur avec paramètres ***(0.5pt)***
- Une méthode ***equals*** : qui vérifie l'égalité en se basant sur le nom, la marque et le nombre de siège. ***(1pt)***

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

4- Classe « **Camion** » qui hérite de Vehicule. Elle possède :

- Deux attributs supplémentaires : **capacité** *(double)*, **type de capacité** *(tonne, litre)* ***(0.5pt)***
- Un constructeur avec paramètres ***(0.5pt)***
- Une méthode **equals** qui vérifie l'égalité en fonction de la marque, la capacité, et le type de capacité. ***(1pt)***

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

5- Classe « **ParcAuto** » : l'entreprise donne à chaque véhicule un identifiant unique interne. A cet effet, cette classe possède :

- Un attribut privé « **mapVehicules** » : c'est une *map* des véhicules dont la clé est un long qui représente l'identifiant du véhicule ; et la valeur est un objet de type « **Vehicule** ». ***(0.5pt)***
- Constructeur par défaut qui initialise mapVehicules par un ***HashMap (0.5pt)***
- Les méthodes suivantes :
  - **public boolean addVehicle(Long id, Vehicule ve)** : la méthode doit ajouter le véhicule *ve* à la map et retourne true si on n'a pas encore dépassé la capacité du parc. Sinon elle doit lever l'exception *OutOfCapacityException* ***(1.5pt)***
  - **public boolean removeByImmatricule(String im)** : la méthode doit chercher le véhicule ayant l'immatriculation im, le supprimer et retourner true. S'il n'existe aucun véhicule ayant l'immatricule *im*, la méthode retourne false ***(1pt)***
  - **public Set<Vehicule> toSet()** : permet de retourner un set contenant tous les véhicules qui se trouvent dans le map ***(1pt)***
- Les méthodes suivantes basées sur les streams :
  - **public long nombreVehicules() :** retourne le nombre de véhicules fabriqués en 2024 et ayant le type de carburant « diesel ». ***(1pt)***
  - **public double getTotalCapacite() :** retourne *la capacité totale en tonnes* de tous les camions dans le parc *(ceux dont le type de capacité égale au tonne)*. ***(1pts)***
  - **public void getMaxCapacite()** : affiche les informations du camion ayant la plus grande capacité ***(2pts)***
  - **public void afficherVehicules(String marque)** : affiche l'immatriculation en ordre alphabétique des véhicules ayant la marque passée en paramètres ***(2pts)***

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

Indications :

- Utiliser soit ***getClass***() ou ***instanceof*** pour vérifier le type des objets
- Utiliser le « ***caste*** » pour convertir le type d'un objet à un autre

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**